HashCompanion

~A Web3 Companion for the Hedera Ecosystem~

by **rocker_bell8** for the Hedera Ecosystem. Apex Hackathon

The future runs on Hedera.

Summary

- Introduction
- Problem
- Solution
- Technologies Used
- Mind Graph
- Roadmap

Introduction:

Abstraction**, HaschCompanion** is a Hedera-built Web3 dApp companion application designed to help users explore and interact with the Hedera ecosystem directly from their local machine.

The application includes a browser extension tab that simplifies the user experience and helps bridge the gap between technical and non-technical users. The idea for HaschCompanion originated from the challenges our team, **HashForce**, faced while competing in previous hackathons (Origins, Trilogy, and Ascension). Those experiences highlighted several obstacles when trying to quickly interact with the network and test concepts during development.

HaschCompanion embodies multiple core concepts of blockchain technology, particularly **traceability and security**, while presenting them through an intuitive and visually appealing interface built around a companion-style concept application.

Through our previous participation and experience, we realized that establishing a connection with the network

through a clear and efficient interface is crucial. By leveraging the **Hedera Hashgraph SDK** within our extension application, we created a solution that maintains its own interface while seamlessly interacting with the Hedera network.

With HaschCompanion, users can create accounts, connect with their existing accounts, and explore the Hashgraph network. The application also allows users to interact with the network through a set of basic tools and functionalities, making the Hedera ecosystem more accessible through our **Hashgraph Companion**.

Problem

Interacting with the **Hashgraph network and technology** requires the creation of a **Hedera EVM-compatible account**, which differs from a typical EVM account used in most blockchain ecosystems. In order to interact with smart contracts, perform transactions, and execute operations on the network, users also need a **signer** to authorize and validate those actions.

During our development journey, we encountered several challenges while working with the testnet accounts provided through the Hedera portal. One of the most noticeable issues was the **limited availability of wallets**

and tools that allow developers and users to easily create accounts, connect to the network, and interact with the Hedera ecosystem.

This raised an important question for our team: *why not apply the knowledge and experience we had accumulated and tackle this challenge ourselves?* What began as a simple idea quickly turned into a journey of **exploration, experimentation, and innovation**.

Through this process, we were able to make significant progress and develop **HashCompanion** — a tool designed to simplify access to the Hashgraph network and make interacting with the ecosystem more intuitive.

HashCompanion is your companion for navigating and interacting with the Hashgraph network.

Solution:

Account Security and Ownership

Your **account is everything**. Without your account, you lose access to your data and your ability to interact with the ecosystem. With this philosophy in mind, we emphasize the importance of keeping your account **private and secure**.

Our dApp **does not store or send your account data to any external source**. Since blockchain accounts are immutable and cannot be altered, the only credentials required are your **private key or seed phrase**. These are used solely to import your account and connect to the network. In our case we require your hedera created AccountID and privateKey, accounts can be of type **ED25519** Or **ECDSA.**

NB: we leverage the creation of ECDSA in our proof of concept and we also charge the account with 25Hbar testnet balance to test our use Case.

Connection to the network is established by verifying the account through the Hedera network and executing a **small transaction fee** required for interaction.

With your account connected, you gain access to a growing ecosystem of applications within **HashCompanion**, including:

- TodoApp
- MailerApp (Messaging)
- DexScanner
- HCAccountManager
- AICompanion

All these applications are built on the **Hedera network** using the **Hedera SDK**.

Applications Inside HashCompanion

TodoApp — TodoList

The **TodoApp** allows connected users to manage tasks in a decentralized way.

Features include:

- Adding new todos
- Changing the status of a todo between **Active** and **Completed**
- Each active todo has a **1-week expiration timestamp**
- Updating the status (complete → active) can **extend the time before expiration**

The goal of this system is to make task management **interactive while also encouraging users to stay consistent with their tasks**.

MailerApp — Decentralized Messaging

The **MailerApp** enables users to send messages through the network.

Features:

- Users must first **register into the contract domain**
- Registered users can send messages to other registered users
- Messages are sent with a **small network fee**
- Delivery is **fast and near-instant**

This creates a simple **blockchain-based messaging system** inside the ecosystem.

DexScanner — Blockchain Interaction Explorer

The **DexScanner** allows connected users to explore blockchain activity.

Users can:

- View their **personal interaction history** with the Hedera blockchain
- Search interactions using **transaction IDs**
- Explore **account activity by account ID**
- Browse interactions on both **Testnet and Mainnet**

This tool acts as a **lightweight blockchain explorer integrated directly into the application**.

HCAccountManager — Account Management

The **HCAccountManager** allows users to import and manage multiple Hedera accounts.

Features include:

- Importing Hedera accounts into HashCompanion
- Secure storage in **Chrome local storage**
- Easy **switching between multiple accounts**

This simplifies account management for users who interact with the network frequently.

AICompanion — AI Integration for Web3

AICompanion is the latest concept application integrated into HashCompanion.

It allows users to interact with an AI assistant directly from the Web3 environment. The current implementation has been tested with the **Gemini API**, and it performs reliably.

Future updates will include:

- A **credit-based system** where users earn credits for interacting with the AI
- Credits can then be used to **continue using AI services within the platform**
- Support for **BYOK (Bring Your Own Key)** so users can connect their preferred AI agent APIs

- A **registry of available AI agents** that can be accessed:
  - For free
  - Through subscription
  - Through the credit system

This creates a flexible **AI-powered companion layer inside the Web3 ecosystem**.

Technology Stack Used in HashCompanion

HTML (HyperText Markup Language)

HTML is the foundation of the web interface. It structures the content of the application and defines the layout of the user interface.

Use in our project:

- Creating the structure of the extension UI
- Organizing elements such as forms, buttons, and containers
- Providing the base layout for components like the TodoApp, MailerApp, DexScanner, and Account Manager

HTML acts as the structural backbone of the HashCompanion interface.

CSS (Cascading Style Sheets)

CSS is used to style the interface and control the visual appearance of the application.

Use in our project:

- Designing the layout of the extension popup and pages
- Styling components for a clean and intuitive UI
- Managing responsive layouts and visual hierarchy
- Improving usability and making the application visually appealing

CSS ensures the application remains user-friendly and visually engaging.

JavaScript / TypeScript

JavaScript is the main programming language used to build the application's logic and interactivity.
TypeScript is a strongly typed version of JavaScript that helps improve code reliability and maintainability.

Use in our project:

- Handling user interactions
- Managing application state and logic

- Connecting the front-end interface to blockchain services
- Processing transactions and contract calls
- Improving code safety and scalability through TypeScript typing. This layer powers the core logic of HashCompanion.

React

React is a JavaScript library used to build modern user interfaces through reusable components.

Use in our project:

- Creating modular UI components
- Building the different application modules such as:
  - TodoApp
  - MailerApp
  - DexScanner
  - Account Manager
  - AICompanion
- Managing dynamic updates when user data changes

React allows the project to remain scalable and maintainable.

ReactDOM

ReactDOM is responsible for rendering React components into the browser's DOM (Document Object Model).

Use in our project:

- Rendering the React application inside the Chrome extension interface
- Updating UI components dynamically when the state changes

It acts as the bridge between React components and the browser UI.

React Router DOM

React Router DOM enables navigation between different pages inside the application without reloading the page.

Use in our project:

- Creating navigation between modules such as:
  - TodoApp
  - MailerApp
  - DexScanner
  - Account Manager

    - AICompanion
- Managing routes within the extension interface

This allows the application to behave like a multi-page Web3 dashboard.

Hedera Hashgraph SDK

The Hashgraph SDK is the official developer toolkit for interacting with the Hedera network.

Use in our project:

- Creating Hedera accounts
- Connecting users to the network
- Sending transactions
- Interacting with smart contracts
- Querying blockchain data

This SDK is the core connection layer between HashCompanion and the Hedera network.

Ethers.js

Ethers.js is a JavaScript library used to interact with Ethereum-compatible smart contracts and blockchain environments.

Use in our project:

- Interacting with EVM-compatible smart contracts on Hedera
- Managing wallet signing and contract calls
- Encoding and decoding smart contract data

It helps integrate EVM contract interaction inside the Hashgraph ecosystem.

Solidity

Solidity is the programming language used to write smart contracts for EVM-compatible blockchains.

Use in our project:

- Developing smart contracts for:
  - TodoApp
  - MailerApp messaging system
  - User registration
- Defining decentralized logic executed on the blockchain

Solidity provides the decentralized backend logic for our dApps.

Chrome Storage & LocalStorage

These technologies allow the browser extension to store data locally on the user's device.

Chrome Storage

Chrome Storage API is designed specifically for browser extensions.

Use in our project:

- Storing imported Hedera accounts securely
- Managing multiple accounts inside the HCAccountManager
- Saving extension settings and session information

LocalStorage

LocalStorage allows persistent data storage inside the browser.

Use in our project:

- Storing UI preferences
- Caching non-sensitive data
- Maintaining session states between extension uses

Google Generative AI (AI Companion Helper)

The **AICompanion** feature inside HashCompanion integrates **Google Generative AI models through the **Gemini API to provide an intelligent assistant within the Web3 environment.

This integration allows users to interact with an AI assistant directly from the application while performing blockchain-related tasks.

Purpose in HashCompanion

The AICompanion acts as a **smart assistant for the user**, helping them navigate the Hedera ecosystem and interact more efficiently with the application's features.

Instead of requiring users to understand every technical detail of the network, the AI assistant provides **guidance, explanations, and contextual help**.

How It Works

The AICompanion connects to the **Gemini API** provided by Google's generative AI platform. When a user sends a prompt through the AI interface:

1. The request is sent to the Gemini API.
2. The model processes the prompt using its generative language capabilities.
3. The response is returned and displayed inside the AICompanion interface.

This allows the application to provide **dynamic AI-powered interactions** within the extension.

Features of AICompanion

Current capabilities include:

- Interactive **AI chat interface**
- Assistance navigating HashCompanion tools
- Help understanding **Hedera network operations**
- Guidance for interacting with the TodoApp, MailerApp, and DexScanner
- General Web3 and blockchain support

The system has been tested using the **Gemini API**, which currently provides fast and reliable responses inside the application.

Future Development

Future versions of AICompanion will expand its capabilities through:

Credit-Based AI Usage

Users will receive **credits for interacting with the AICompanion**. These credits can then be used to access AI features inside the platform.

BYOK (Bring Your Own Key)

Users will be able to integrate their own AI providers by supplying their personal API keys. This allows flexibility to connect different AI agents or services.

AI Agent Marketplace

HashCompanion aims to introduce a registry of **AI agents** that users can access:

- Free agents
- Subscription-based agents
- Agents powered by the credit system

Role in the HashCompanion Ecosystem

The AICompanion represents the **intelligent layer of the application**, combining **AI assistance with Web3 tools**.

By integrating generative AI technology, HashCompanion becomes more accessible to both:

- **technical users** (developers and Web3 enthusiasts)
- **non-technical users** exploring the Hedera ecosystem

This approach helps bridge the gap between **blockchain complexity and user-friendly interaction**.

Summary

Together, these technologies enable HashCompanion to function as a Web3 companion application and browser extension that allows users to:

- Create and manage Hedera accounts
- Interact with smart contracts
- Send messages and manage tasks
- Explore blockchain activity
- Integrate AI tools within the Web3 ecosystem

The combination of modern web technologies, blockchain SDKs, and browser extension tools creates a powerful environment for interacting with the Hedera Hashgraph network directly from the user's browser.

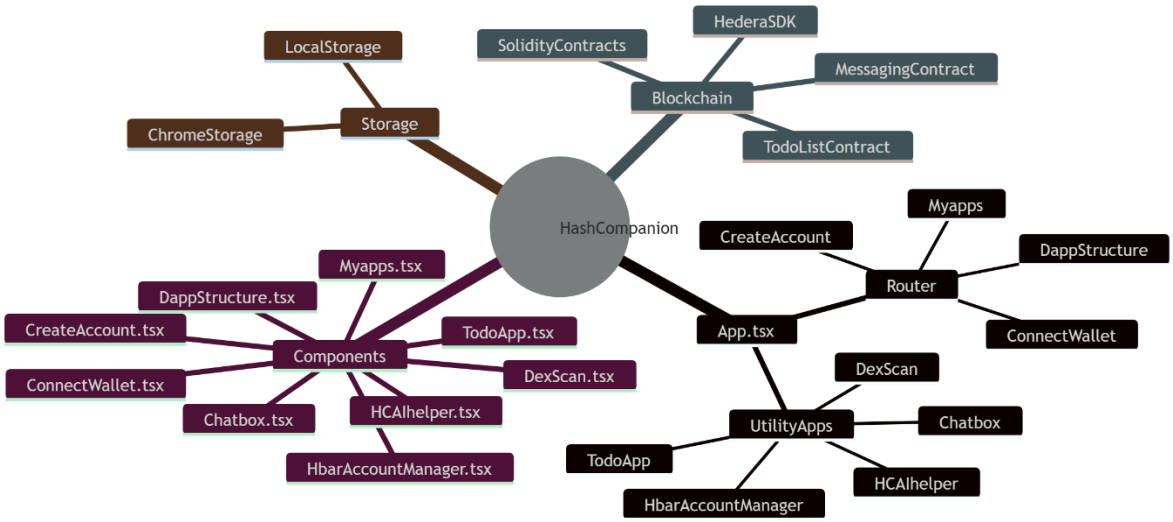
MINDMAP

Below is a UML-style mind graph using Mermaid that describes the routing logic of App.tsx and the overall navigation flow of your HashCompanion extension.

This diagram focuses on:

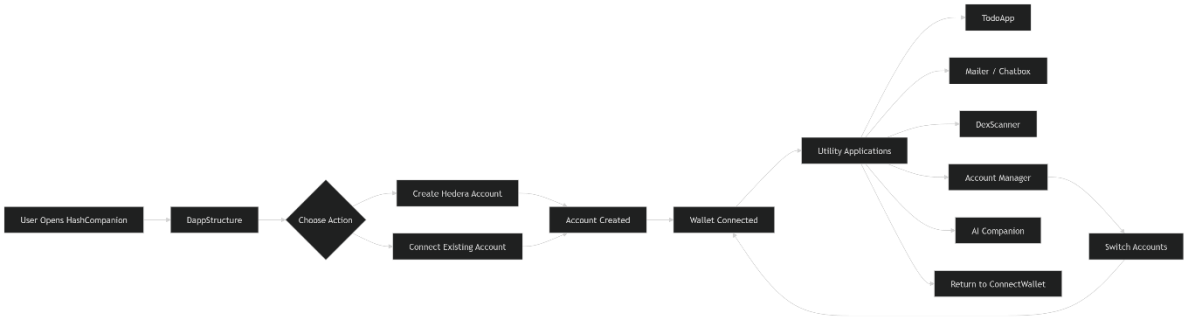
- Router structure
- Wallet state
- Component navigation
- Utility dApps
- Account manager logic

HashCompanion Router & App Architecture



High-Level User Flow

Here is the **conceptual flow of the application**.



Architecture Summary

HashCompanion architecture consists of four main layers:

- **UI Layer**

React components:

- DappStructure
- CreateAccount
- ConnectWallet
- MyApps
- TodoApp
- Chatbox
- DexScan
- HbarAccountManager
- HCAIhelper

Routing Layer

Managed by **React Router**

Routes:

/

CreateAccount

ConnectWallet

Myapps

todoApp

chatbox

DexScan

HCmanager

HCAIhelper

Wallet & State Layer

Handles:

- **Hedera account connection**
- **Private key handling**
- **EVM address generation**
- **Account switching**

Key functions:

connectAccount()

clearAccount()

handleUseWallet()

Blockchain Interaction Layer

Powered by **Hedera SDK**

Operations:

- Account connection
- Balance queries
- Transaction signing
- Smart contract interaction

Roadmap

Phase 1 — Core Foundation (Completed / MVP)

Goal: Build the basic infrastructure for interacting with the Hedera network.

Key milestones:

- Develop the Chrome extension architecture
- Implement account creation using Hedera SDK
- Enable wallet connection with private key
- Generate EVM-compatible addresses
- Implement local account storage using Chrome Storage
- Build the React-based UI framework
- Integrate React Router navigation
- Deploy the initial TodoApp smart contract
- Enable basic Hedera network interaction

Outcome:
A working MVP that allows users to create, connect, and interact with the Hedera network.

Phase 2 — Utility Applications (Current Development)

Goal: Expand the ecosystem with useful decentralized tools.

Key milestones:

- Launch TodoApp decentralized task manager
- Build MailerApp / Chatbox for blockchain messaging
- Develop DexScanner for transaction and account history
- Implement HCAccountManager for managing multiple accounts
- Improve UI/UX design for the extension
- Optimize transaction feedback and notifications

Outcome:
Users gain access to a suite of practical Web3 utilities within the HashCompanion interface.

Phase 3 — AI Integration

Goal: Introduce intelligent assistance inside the platform.

Key milestones:

- Integrate Google Generative AI
- Implement the Gemini API
- Build the AICompanion chat interface
- Provide AI assistance for navigating the dApp
- Enable blockchain help and explanations

Outcome:
Users can interact with an AI-powered assistant directly within the Web3 environment.

Phase 4 — Ecosystem Expansion

Goal: Turn HashCompanion into a broader Web3 toolkit.

Key milestones:

- Add DeFi interaction tools
- Integrate NFT viewing and management
- Expand DexScanner analytics
- Enable smart contract deployment tools
- Improve cross-application interaction

Outcome:
HashCompanion evolves into a multi-tool Web3 dashboard for the Hedera ecosystem.

Phase 5 — AI + Web3 Economy

Goal: Create a decentralized AI ecosystem within the application.

Key milestones:

- Introduce AI credit reward system
- Allow BYOK (Bring Your Own Key) for AI providers
- Build an AI agent marketplace
- Enable subscription-based AI services

- Integrate on-chain credit tracking

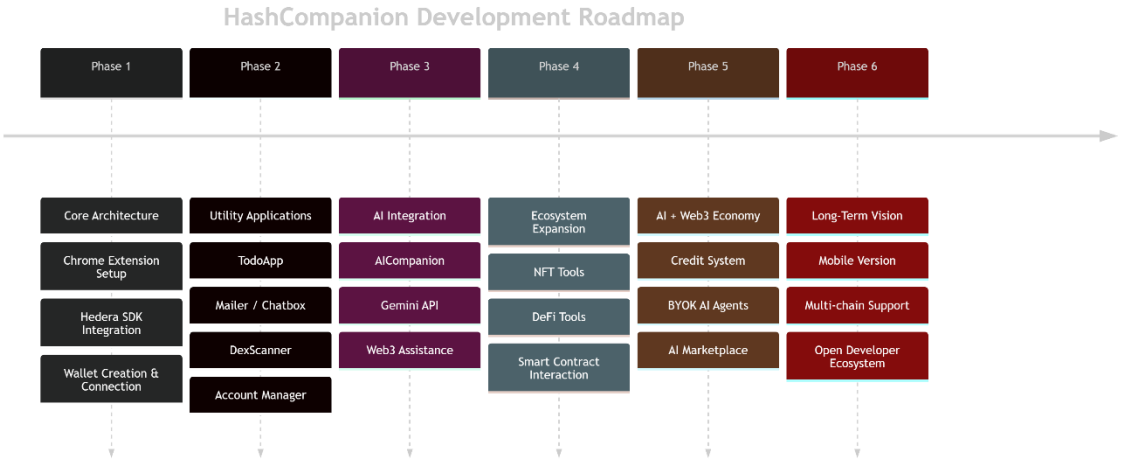
Outcome:
A hybrid AI + Web3 platform where users can interact with AI services directly inside the blockchain ecosystem.

Phase 6 — Long-Term Vision

Goal: Establish HashCompanion as a universal Hedera companion tool.

Future directions:

- Mobile version of HashCompanion
- Multi-network support
- Plugin system for developers
- Open-source contributor ecosystem
- Community-built dApps inside HashCompanion

Outcome:
HashCompanion becomes a gateway for interacting with the Hedera network and Web3 applications.

HashCompanion